TripCraft Pro — Hints

> Read the story and try to build it first. This project **wraps your finished TripCraft (4.1)** — make sure that runs before you start. These hints point you at the pieces and the easy-to-miss parts. The full code is in the complete solution.

Hint 1 — the pieces, and the one rule

Copy TripCraft's `src/`, `data/`, `main.py` in **unchanged**, then add four things *around* them — never inside them:

- `src/pipeline.py` — `run_pipeline(config, destination)`: the single door into the team. Runs the graph, returns a plain `dict` (`destination`, `status`, `itinerary`). Both the API and the terminal call this.
- `src/api/` — `schemas.py` (Pydantic request/response), `routes.py` (the three endpoints), `main.py` (the FastAPI app uvicorn starts).
- `src/db/` — `models.py` (`RunRecord`), `database.py` (`init_db`, `save_run`, `get_run`) over SQLite.
- `src/evals/` — `tasks.py` (fixed destinations + checks), `judge.py` (LLM judge score), `run_eval.py` (pass rate), `regression_check.py` (break a prompt, confirm the drop).
- `Dockerfile` + `.dockerignore`.

The rule: **the agents don't change.** If you find yourself editing `src/agents/`, stop — the production layer goes around the team.

Hint 2 — the pipeline and the database

The pipeline turns the graph's state into a storable dict:

```
def run_pipeline(config, destination):
    final = app.invoke(start_state(destination))   # app built once
    itinerary = ""
    if final["status"] == "done":
        itinerary = final["draft"]
    return {"destination": destination, "status": final["status"],
            "itinerary": itinerary}
```

(Write the `if` as two lines if you're avoiding ternaries.) The DB is a tiny SQLite layer: `init_db()` creates a `runs` table (`id` autoincrement, destination, status, itinerary, created_at); `save_run(result)` inserts and returns `lastrowid` ; `get_run(id)` selects one row (or `None`). Keep SQL in `db/` only — routes call `save_run` / `get_run` , never SQL.

Hint 3 — the three routes, and the one import trick

```
from src import pipeline          # NOT: from src.pipeline import run_pipeline
...
result = pipeline.run_pipeline(config, request.destination)
```

Importing the **module** (`from src import pipeline`) and calling `pipeline.run_pipeline(...)` is what lets a test swap the pipeline for a fake one (`pipeline.run_pipeline = fake`) with no model and no key — the route looks the function up on the module at call time. If you `from src.pipeline import run_pipeline` , the route binds the real function at import and the test can't replace it.

Routes: - `GET /health` → `{"status": "ok"}` . - `POST /itineraries` → validate with `ItineraryRequest` , run the pipeline, `save_run` , return `ItineraryResponse(id=..., **result)` . - `GET /itineraries/{id}` → `get_run` ; `None` → `raise HTTPException(status_code=404, ...)` .

An **unknown** destination is a normal `200` with `status="unknown"` , stored like any other run — it isn't an error.

Hint 4 — the eval suite, and proving it catches a regression

The judge scores a *finished* itinerary against written checks with structured output (same idea as the Reviewer, but on the output):

```
chain = prompt | model.with_structured_output(JudgeScore)   # score 1-10
```

`run_eval` runs the pipeline over the fixed tasks, judges each, and prints the **pass rate** (share at or above the bar). The regression check is the important one — it proves the suite actually *detects* worse output:

```
good = run_suite()
# monkeypatch the prompt loader so the writer prompt is sabotaged:
prompts.load_prompt = sabotaged_loader
bad = run_suite()
assert bad < good          # the suite caught the regression
```

Because `run_pipeline` reads the writer prompt through `load_prompt`, swapping that function makes the team produce worse itineraries, the judge scores them lower, and the pass rate falls — which is exactly what would block a bad release in CI. Restore the real loader afterward.

Still stuck? The whole production layer, every file, is in the complete solution.